

Serialization of Concurrent Operations in a Distributed Collaborative Environment

Maher SULEIMAN, Michèle CART, Jean FERRIÉ

LSI, University of Montpellier II
860, rue de St Priest, 34090 Montpellier
e-mail: <Lastname>@lsi.fr

ABSTRACT

In a distributed groupware system, objects shared by users are subject to concurrency and real-time constraints. In order to satisfy these, various concurrency control algorithms [4] [11] have been proposed that exploit the semantic properties of operations. By ordering concurrent operations, they guarantee consistency of the different copies of each object. The drawback of these algorithms is that in some situations they can result in inconsistent copies, a non-respect of user's intentions, and in the need to undo and redo certain operations.

The principle objective of this paper is to present an algorithm that overcomes these problems. The algorithm is based on the notion of user's intention, and also on the construction of equivalent histories by exploiting and combining some general semantic properties such as forward/backward transposition.

Keywords: groupware systems, concurrency control, distributed systems, multi-user editors, operation transposition.

1. INTRODUCTION

The objective of a collaborative environment is to facilitate team working and, in particular, to enable a group of persons to manipulate shared objects, and modify them in a coherent manner. There are many collaborative activities, examples being simultaneous writing of a document by different authors, or cooperative design. In general, an object involved in a collaborative activity is submitted to concurrent accesses and real-time constraints. The real-time aspect necessitates every user seeing the effects of his own acts on the object immediately, and the effects of the acts of other users almost immediately. Furthermore, a collaborative environment is often a distributed one. Consequently, the problem is to conciliate both real-time and consistency constraints, as the object may be modified concurrently by many users. To satisfy these requirements, it is necessary [5] [8] that concurrency control does not use a blocking protocol which could defer user actions.

Some collaborative systems [9] [3] [16] use locks for concurrency control, thus preventing simultaneous access to the object. Other systems [11] [4] [18] use multiple copies to permit concurrent access to the same object, and use semantic properties of the operations defined on the object

to reorder concurrent operations while preserving object consistency.

More precisely, the works described in [4] use a transformation property to construct the history associated with each copy of the object. The histories of the different copies of the object are guaranteed to be equivalent (i.e. lead to the same final state), although the order of concurrent operations can be different. However, the problem of partial concurrency evoked in [1], that can lead to incoherence, is not solved. The works described in [11] use commutativity and masking properties to reorder the histories associated with object copies so as to respect a unique order defined *a priori* by timestamps. Unfortunately, in the absence of commutativity and masking properties, some operations have to be undone and then redone. In addition, the user intention is not always respected. In [18], the transformation property is used in order to respect the user intention. Moreover, the histories of the different copies of the object are ordered by using the same order to guarantee consistency of the object. But unfortunately, as in [4], the problem of partial concurrency is not solved and as in [11], operations have to be undone and then redone in order to respect the same order.

In this paper we aim to design a system that does not have these drawbacks. Our objective is to satisfy both concurrency and real-time constraints, while solving the partial concurrency problem and respecting the user intention without undoing and redoing operations.

The paper is organised into two sections. The first section presents a solution adapted to a collaborative environment which we call centralised, where a unique copy per object is maintained. This solution is based on the notion of intention, and uses forward transposition. The second section presents the distributed solution with a copy of each object per site. It details the principles on which the construction of equivalent histories is based. Then, algorithms that use forward as well as backward transposition and causal broadcast are presented.

2. CENTRALISED ENVIRONMENT

2.1 Model and Problematic

In a centralised environment, a collaborative system is constituted of a set of user sites (one user per site) connected by a reliable network to a server site. The server site is responsible for the storage of the objects (i.e. documents, texts, graphics, ...) shared by the users collaborating in a common activity.

Each object O may be manipulated by the users by means of specific operations $\{op_1, op_2, \dots, op_n\}$ that must be

Permission to make digital/hard copies of all or part of this material for personal or classroom use is granted without fee provided that the copies are not made or distributed for profit or commercial advantage, the copyright notice, the title of the publication and its date appear, and notice is given that copyright is by permission of the ACM, Inc. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires specific permission and/or fee.

GROUP 97 Phoenix Arizona USA

Copyright 1997 ACM 0-89791-897-5/97/11..\$3.50

transmitted to the server to be executed. We suppose that the state (i.e. the representation) of each object stored on the server is visible, at each instant, by a user inside a window on his own site.

The progress of an operation is decomposed into four steps: *generation*, *sending*, *reception* and *execution*. Thus the operation is first *generated* on the user site and *sent* to the server; it is then *received* by the server and *executed* on the object. We suppose (without losing generality) that the results of an operation executed by the server on an object are immediately visible to all users sharing that object.

In order to obtain the maximum of concurrency and to preserve the collaborative aspect of the work made by users manipulating the same object, our model does not impose any locking protocol for controlling concurrency between operations. We will study the consequences of this approach, when several users modify the same object at the same instant.

Let us specify the causality relation that may exist between two operations op_1 and op_2 , respectively generated by users U_1 and U_2 .

Definition 1. Causal precedence between two operations. We say that op_1 *precedes causally* op_2 , noted op_1 *precedes* op_2 , (Fig. 1A) if op_2 has been generated after that the results of op_1 have become visible to user U_2 . In this case, the effects of op_1 are implicitly taken into account by op_2 .

Definition 2. Concurrent operations.

Two operations op_1 and op_2 , respectively generated by users U_1 and U_2 are said to be *concurrent* (Fig. 1B) iff: not (op_1 *precedes* op_2) and not (op_2 *precedes* op_1).

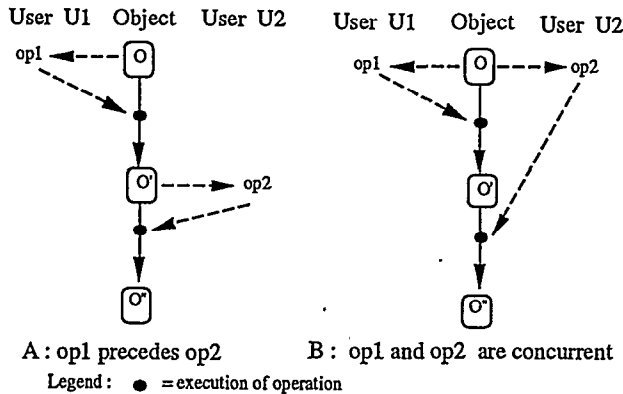


Figure 1. Causal precedence and concurrency between operations.

Example 1. Collaborative text editor.

Let a text be composed of sentences. Each sentence is an object represented by a string of characters. Operations defined on this object are *insert*(p, c) and *delete*(p) where:

insert(p, c) : inserts the character c at the position p (before p^{th} character);

delete(p) : delete the p^{th} character.

Suppose that both users U_1 and U_2 work simultaneously on the same object, whose state O is "on systems ...". Suppose that they want to change O into $O' = \text{"one system ..."}.$ User U_1 puts the cursor after 'on' and then inserts the character 'e'. The operation generated by site 1 is $op_1 =$

$\text{insert}(3, 'e')$. On the other side, the action of user U_2 to delete the last 's' in the word 'systems' is translated into the operation $op_2 = \text{delete}(10)$.

Operations op_1 and op_2 are concurrent (they are defined on the same state), but they will be executed by the server serially. If the server executes them as they are, in the reception order ($op_1; op_2$), the final state will be $O' = \text{"one systes ..."}.$ Notice that this state does not correspond to the intention of user U_2 (Fig. 2A). To obtain the result desired by user U_2 , the server must not execute op_2 but another operation $op_2' = \text{insert}(12, 's')$ obtained from op_2 by considering that op_1 has been executed before it (Fig. 2B).

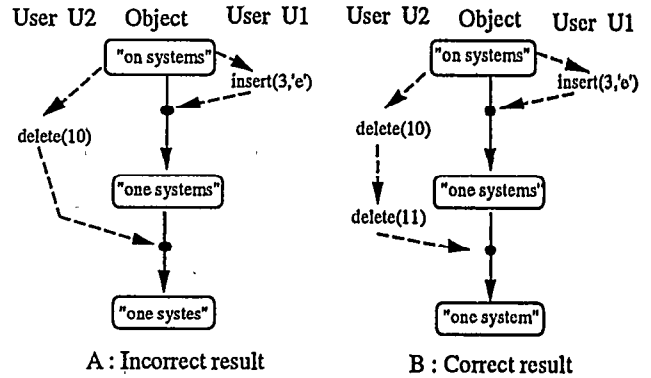


Figure 2. Execution of concurrent operations.

Thus, the problem is to design a method able to integrate the effects of op_2 , to the new state O' . More precisely, the problem is to execute concurrent operations serially while respecting the intentions of the users. Thus, the server may be led to transform a received operation, before executing it. This transformation requires the object history to be known in order to determine which operations are concurrent.

Definition 3. Object history.

The history $H(n)$ of an object, is the sequence, noted $op_1.op_2 \dots op_n$, of n operations executed on the object and ordered in their execution order.

In short, construction of a history $H(n)$ is not a simple sequential juxtaposition of operations received by the server. In fact, some concurrent operations must be transformed before being executed and added to the history. Thus, suppose that the object is in state O_n , after executing the sequence $op_1.op_2 \dots op_n$. Its history is $H(n) = op_1.op_2 \dots op_n$. When the server receives operation op , generated from the object state O_i (with $O_i \neq O_n$), the problem consists of finding an operation op_{n+1} defined on the state O_n , that produces the same effects as op .

Notations

$O_0, O_1, \dots, O_i, O_n, O_f$ designate states of object O .

$O_i.op = O_f$: means that the execution of the operation op has changed object state from O_i to O_f .

$O_0.op_1.op_2 \dots op_n = O_f$: means that the execution, in that order, of the sequence $op_1.op_2 \dots op_n$ has changed object state from O_0 to O_f .

In the following, we need to compare the effects of two sequences of operations on an object.

Definition 4. Equivalence of two sequences of operations.

Two sequences of operation seq_1 et seq_2 are equivalent (noted $seq_1 \equiv seq_2$) if, starting from the same state, they lead the object to the same state, that is to say:

$$\forall O_i, O_i.seq_1 = O_i.seq_2.$$

2.2 Serialization of Concurrent Operations Respecting the Intentions of the Users

Intention of the User

Consider the object O whose state is the string of characters "system". When a user wants to add the letter 's' at the end of this word, he puts the cursor after the 'm' and he enters an 's'. The editor translates this action into the operation $insert(7, 's')$ which inserts 's' at position 7. We say that the user intention is {add 's' at the end of 'system'} and this intention is realized by the operation $insert(7, 's')$.

The intention is an abstraction of the result to be obtained. The operation that realizes it must take the object state into account. So, if the state of the object O were "the system of...", the same intention would be realized by the operation $insert(11, 's')$.

In order to distinguish the intention from the operation that realizes it, we will use the following notations:

$Operation(I, O_i)$: designates the operation that realizes the intention I on the object in the state O_i ;

$Intention(op, O_i)$: expresses the user intention realized by the operation op executed on the object in the state O_i .

For example:

$Operation(\{add 's' after 'system'\}, "system of") = insert(7, 's'),$
 $Intention(insert(7, 's'), "system of") = \{add 's' after 'system'\}.$

The preceding notations can be generalized to a series of intentions corresponding to a sequence of operations. This could be defined recursively by:

$$Intention([op_1.op_2....op_n], O_i) = Intention(op_1, O_i).Intention([op_2....op_n], O_i.op_1)$$

Thus, to express that a sequence of operations $[op_1.op_2....op_n]$, executed in that order on O_i , realizes the series of intentions $I_1.I_2....I_n$ we will write:

$$Intention([op_1.op_2....op_n], O_i) = I_1.I_2....I_n$$

The following abbreviated notation will be also used:
 $O_i.op_1.op_2....op_n \rightarrow I_1.I_2....I_n.$

Serialization of Two Concurrent Operations

Let us take Example 1 and interpret it in the light of what has been said. Users U_1 and U_2 see simultaneously the object in the state $O = "on systems..."$ and each one has a different intention:

$I_1 = \{insert 'e' after 'on'\}$
 $I_2 = \{delete the last 's' of the word 'systems'\}.$

These intentions are realized by the operations:

$op_1 = Operation(I_1, O) = insert(3, 'e')$
 $op_2 = Operation(I_2, O) = delete(10).$

After being generated and sent to the server, these two concurrent operations will be received and executed serially by the server. If op_1 is executed first, the new state of the object is $O' = "one systems..."$. The intention I_1 is thus realized. Then, when op_2 is executed on the state O' , the resulting new state is $O'' = "one systes..."$. In this case the intention I_2 is not respected (Fig. 2A). In fact, in order to respect the intention I_2 it would be necessary to execute (Fig. 2B) $op_2' = delete(11)$ instead of $op_2 = delete(10)$. We would obtain:

$O.op_1.op_2' \rightarrow I_1.I_2$ with:

$op_2' = Operation(I_2, O.op_1), I_1 = Intention(op_1, O),$
 $I_2 = Intention(op_2', O.op_1) = Intention(op_2, O).$

In reality, the server does not know either I_1 nor I_2 , which are abstractions specific to the users. However, from concurrent operations op_1 and op_2 (that realize respectively I_1 and I_2), the server must determine the operation op_2' to be executed on the object state $O' = O.op_1$ in order to respect the intention I_2 . This determination is possible if we define a function of transposition [4], specific to the application and to the semantic of the operations, that gives for each ordered couple of operations (op_1, op_2) an operation op_2' realizing the same intention as op_2 , and that takes the effects of op_1 into account. This function is named *forward transposition* and noted *Transpose_fd*.

Definition 5. Forward transposition.

The forward transposition function, *Transpose_fd* (op_1, op_2), delivers as a result an operation noted $op_2^{op_1}$ and named forward transposed operation of op_2 with regard to op_1 , such that:

$$\forall O_i, Intention(op_2^{op_1}, O_i.op_1) = Intention(op_2, O_i).$$

In Example 1, the transposition to be used is:

$$Transpose_fd(insert(4, ' '), insert(11, 's')) = insert(12, 's').$$

For the operations *insert* and *delete* the corresponding forward transpositions are defined as follows:

$$Transpose_fd(insert(p_1, c_1), insert(p_2, c_2)) =$$

case $p_1 ? p_2$ of

$p_1 < p_2$: return $insert(p_2 + 1, c_2);$

$p_1 > p_2$: return $insert(p_2, c_2);$

$p_1 = p_2$: if $c_1 = c_2$ then return id
else return $insert(p_2, c_2);$ endif

endcase;

$$Transpose_fd(delete(p_1), insert(p_2, c_2)) =$$

if $p_1 < p_2$ then return $insert(p_2 - 1, c_2)$

else return $insert(p_2, c_2);$ endif

$$Transpose_fd(insert(p_1, c_1), delete(p_2)) =$$

if $p_1 \leq p_2$ then return $delete(p_2 + 1)$

else return $delete(p_2);$ endif

$$Transpose_fd(delete(p_1), delete(p_2)) =$$

case $p_1 ? p_2$ of

$p_1 < p_2$: return $delete(p_2 - 1);$

$p_1 > p_2$: return $delete(p_2);$

$p_1 = p_2$: return id

endcase;

Notice that when the operations are identical, the forward transposed operation is the null operation (noted *id*). For example, this case happens when two users insert the same

character, in the same position, at the same moment. One operation must be executed and the other one must be ignored; only one operation is sufficient to satisfy the intention of both users.

Serialization of an Operation Concurrent to n Operations

Consider the sequence seq_n of n operations (with $seq_n = op_1.op_2.... op_n$) executed in that order on object O from the state O_i . The resulting state is O_f such as $O_i.seq_n = O_f$. Let op be a new operation generated from the state O_i and respecting intention I . We say in this case that op has been generated concurrently with the sequence seq_n . As the object state has changed from O_i to O_f , it is no longer possible to execute op on O_i . So, it is necessary to define and execute an operation that realizes I on O_f .

By noting op^{op1} the forward transposed operation of op with regard to op_1 (i.e. $op^{op1} = \text{Transpose_fd}(op_1, op)$), $(op^{op1})^{op2}$, also denoted $op^{op1.op2}$, expresses that op was first transposed forward with op_1 and the resulting operation was then transposed with op_2 . In the same manner the notation op^{seq_n} designates the forward transposed operation of op with the operations of the sequence seq_n , where $seq_n = op_1.op_2.... op_n$.

Definition 6. Forward transposition with a sequence of operations.

The forward transposed operation of op with the sequence seq_n , noted op^{seq_n} , is the resulting operation of a successive forward transposition of op with the operations of the sequence seq_n . op^{seq_n} is formally defined by the following recursive formula:

$$op^{seq_n} = \text{Transpose_fd}(op_n, op^{seq_{n-1}}) \text{ with: } seq_n = op_1.op_2.... op_n = seq_{n-1}.op_n \text{ and } op^{seq_0} = op.$$

The next lemma expresses that the execution of op^{seq_n} from the state O_f respects the intention I of the operation op .

Lemma 1. If op and seq_n operate on the same object state O_i then: $\text{Intention}(op, O_i) = \text{Intention}(op^{seq_n}, O_i.seq_n)$.

Serialization of n Concurrent Operations

Let $op_1, op_2, ..., op_n$ be n concurrent operations generated from the same object state O_i respectively respecting the n intentions $I_1, I_2, ..., I_n$. The problem is to determine the n operations which must be executed serially from O_i to satisfy the series of intentions $I_1.I_2....I_n$.

The solution consists in successively realizing each one of these intentions: op_1 is executed first to realize I_1 (i.e. $O_i.op_1 \rightarrow I_1$), op_2^{op1} is next executed thus realizing I_2 (i.e. $O_i.op_1.op_2^{op1} \rightarrow I_1.I_2$), then $op_3^{op1.(op_2^{op1})}$, the forward transposed operation of op_3 with $op_1.op_2^{op1}$, is executed to realize I_3 , and so on. More generally, to respect the intention I_j , we must execute the transposed operation of op_j with regard to the operations that have been executed to realize the series of intentions $I_1.I_2....I_{j-1}$. Finally, the operation sequence that respects the n intentions is the following:

$$(op_1).(op_2^{op1}).(op_3^{op1.(op_2^{op1})}).... \\(op_n^{op1.(op_2^{op1})....(op_{n-1}^{op1....})}).$$

To simplify the previous formula, we will use the notation $op_1.op_2.op_3....op_n$, in which the operator $'.'$ preceding an operation signifies that this operation is transposed forward with the operation sequence in the left side of the operator. Formally, this means:

$$seq.op = seq.op^{seq} \text{ where } seq \text{ is a sequence.}$$

Lemma 2. Let n concurrent operations $op_1, op_2, ..., op_n$ such as: $(O_i.op_1 \rightarrow I_1), (O_i.op_2 \rightarrow I_2), ..., (O_i.op_n \rightarrow I_n)$. We have: $O_i.[op_1.op_2: ...op_n] \rightarrow I_1.I_2....I_n$.

Proof. Recursively on n . For $n = 1$, $O_i.op_1 \rightarrow I_1$. Suppose that the sequence $seq = [op_1.op_2: ...op_{n-1}]$ is such that: $O_i.[op_1.op_2: ...op_{n-1}] \rightarrow I_1.I_2....I_{n-1}$ and that $O_i.seq = O_f$. To realize I_n from O_f , we execute op_n^{seq} . Then, according to lemma 1:

$$I_n = \text{Intention}(op_n, O_i) = \text{Intention}(op_n^{seq}, O_i.seq).$$

As $seq.op^{seq} = seq.op = [op_1.op_2.op_3....op_n]$, this implies:

$$O_i.[op_1.op_2: ...op_n] \rightarrow I_1.I_2....I_n \quad \square$$

2.3 Application: Integration of an Operation in the History

Consider an object whose state has evolved from the initial state O_0 to the current state O_n . Let $H(n) = op_1.op_2....op_n$ be its history. When a new operation generated from the state O_i and respecting an intention I is received by the server, the problem is to determine the operation op_{n+1} to be executed on O_n to respect the intention I , that is to say:

$$\text{Intention}(op_{n+1}, O_n) = \text{Intention}(op, O_i) = I.$$

Resolution Method

To each object is associated a timestamp v corresponding to the number of the object version. After each operation execution on an object, the timestamp of this object is incremented by one. So, if initially $v=0$, then v will represent the number of operations executed on the object.

Moreover, the version number of the object from which an operation has been generated is associated with this operation when it is generated. In Figure 3, the version number makes it easy to see that $\text{insert}(4, '')$ precedes $\text{insert}(11, 's')$ and that $\text{insert}(11, 's')$ is concurrent to $\text{insert}(5, 'n')$ and $\text{insert}(6, '')$.

More generally, in the history $H(n)$, we assume that operation op_j has been executed on the state O_{j-1} producing the state O_j . If op has been generated from the state O_i , then all the operations of the sequence $[op_1....op_i]$ precede op while the operations $[op_{i+1}....op_n]$ are concurrent to op .

Since the sequence $[op_{i+1}....op_n]$ is executed from the state O_i , according to lemma 1, in order to respect the intention I of op , op must be transposed forward with this sequence. So we have: $\text{Intention}(op, O_i) = \text{Intention}(op^{op_{i+1}....op_n}, O_i.op_{i+1}....op_n)$. As $O_i.op_{i+1}....op_n = O_n$, then $\text{Intention}(op, O_i) = \text{Intention}(op^{op_{i+1}....op_n}, O_n)$. Thus, the new history will become:

$$H(n+1) = op_1.op_2....op_i.op_{i+1}....op_n.op_{n+1} \\ \text{with: } op_{n+1} = op^{op_{i+1}....op_n}$$

Algorithm

When an operation $\langle op, i \rangle$ generated from the object state O_i , the server transposes it forward with the sequence of the

operations that come after the i^{th} operation in $H(n)$. The resulting operation is then executed and added to $H(n)$.

Notation:

op : received operation, to be integrated,

i : version number associated with op at its generation,

H : object's history : $H[j] = \text{operation that has produced } O_j$,

n : number of operations in $H = \text{current version number}$.

The procedure executed by the server is the following.

procedure *Integrate*(op, i):

begin

for $j := i + 1$ **up to** n **faire**

$op := \text{Transpose_fd}(H[j], op);$

endfor;

execute(op);

$H[n+1] := op; \quad \{ \text{integrate } op \text{ into the history} \}$

$n := n + 1;$

end

Example 2. Suppose that user U sees the object in the state $O_1 = \text{"the system ..."}$ (with the version number $v = 1$) and his intention is to add 's' to the word system. The operation $\text{insert}(11, 's')$ is then generated and sent to the server. Suppose that in the meantime two new operations $\text{insert}(5, 'n')$ and $\text{insert}(6, '')$ have been executed by the server (Fig. 3). When received by the server, $\text{insert}(11, 's')$ will be transposed forward with its two concurrent operations, and finally the resulting operation $\text{insert}(13, 's')$ will be executed.

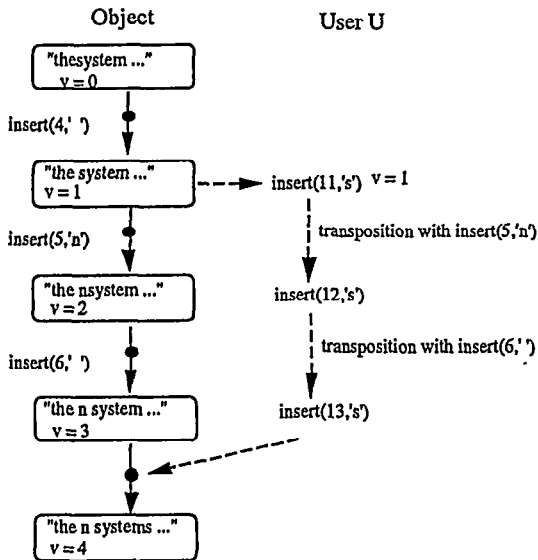


Figure 3. Integration of an operation in the history.

3. DISTRIBUTED ENVIRONMENT

3.1 Model and problematic

A distributed collaborative system is constituted, as in the previous section, by a set of user sites interconnected by a reliable network. The difference is that the shared objects are distributed and replicated. Thus, all sites that have copies of the objects are server sites.

In the following, we suppose that there exists, for each object O , a copy at every site. So each site is both a user and server site. We suppose as well, to simplify the

presentation, that there is one user per site. Each object O may be manipulated by users by means of specific operations $\{op_1, op_2, \dots, op_n\}$. In order to maintain the consistency of the copies of O , every operation intended to be executed on O must be executed on all the copies. For that, an operation generated by a site must be broadcast to all the server sites. The four events associated with an operation are: *generation, broadcast, reception, and execution*.

An operation generated by a site is sent to all sites (including the generator site itself) where it is received and then executed. In order to guarantee users a minimum response time, the execution of an operation on the generator site must not be delayed by the mechanism ensuring the consistency of the copies. More precisely, an operation generated by a site must be executed immediately after its reception by this site. In the following, for a site S , a *local* operation is an operation generated by S , while a *remote* operation designates an operation generated by a site other than S .

In this context, two operations are related by either a *precedence* or a *concurrency* relation.

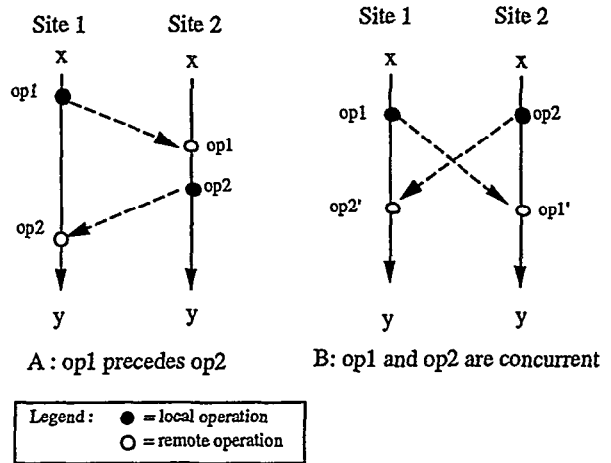


Figure 4. Relation of concurrency and precedence between operations.

Definition 7. Precedence between two operations.

We say that op_1 *precedes* op_2 iff op_2 has been generated by site S after the execution of op_1 on S .

Definition 8. Concurrency between two operations.

We say that op_1 and op_2 are *concurrent* iff: not (op_1 *precedes* op_2) and not (op_2 *precedes* op_1).

The *precede* relation defines a partial order on the operations. This order expresses the potential causality [17] between these operations. If op_1 *precedes* op_2 , op_2 is supposed to be dependent on the effects produced by op_1 . On the contrary, if op_1 and op_2 are *concurrent* it means that these operations are completely independent and that neither one has seen the effects of the other. The relation *precede* must be respected on all sites; in other words, if op_1 *precedes* op_2 , then op_1 must be executed before op_2 on all sites. If op_1 and op_2 are concurrent, they may be executed in any order on different sites. However, the operation which is executed second must be transformed to take the effects produced by the first operation into account and to satisfy the user's intention.

In this context, the problem is to integrate on each site the effects of local and distant operations while respecting the following constraints:

- immediate execution of local operations after receiving them,
- respect of the causality, expressed by the relation precede, between the operations,
- guarantee of the consistency of the copies by respecting intentions associated to the operations.

To each copy of an object O on a site S is associated a history defined as follows.

Definition 9. The history of an object O on a site S , noted $H_{S,O}(n)$, is a sequence of n operations executed on the copy of O on the site S that transformed the object O from its initial state to its current state.

So, the problem is to construct, for every site S and for every object O , a history $H_{S,O}$ respecting the constraints above in such a way that if the object copies were identical at the beginning of the work session, they will also be identical each time the system is quiescent [4], that is to say when there are no more messages circulating in the network.

3.2 Respecting the Causality

To be able to respect the causality between operations we need a technique that allows us to:

- 1) compare two operations to determine if one operation precedes the other,
- 2) deduce, when one site receives an operation, if there exists an operation that precedes it and that has not yet been received.

The technique suggested in [4], like that described in [14], satisfies both these criteria. It uses a state vector for each site that counts operations generated by the other sites and executed on this site. We will adopt this technique to define two procedures for message emission and reception. These procedures guarantee a causal broadcast [17] such as "if the emission of m_1 precedes the emission of m_2 then the reception of m_1 precedes the reception of m_2 ".

State Vector

Let N be the number of sites in the collaborative system. These sites are numbered from 1 to N . On every site S , and for every object O , a vector $V_{S,O}$ composed of N elements is defined where: $V_{S,O}[j]$ = number of operations generated by site j , received by the site S and executed by S on its copy of the object O .

Generation and Broadcast of an Operation

When an operation is generated on a site S from an object O , the current value of the state vector $V_{S,O}$ is associated with it. It is designated by V_{op} . The message broadcast to sites is a triplet: $\langle op, S_{op}, V_{op} \rangle$ where op is the generated operation, S_{op} the generator site and V_{op} the state vector associated with the operation.

The state vectors associated with operations allow us to compare operations to determine whether an operation precedes another one.

To simplify the notation in the next sections, we suppose the existence of only one object. Therefore, the notations V_S (resp. H_S) will be used instead of $V_{S,O}$ (resp. $H_{S,O}$).

Reception

When a site S receives an operation op with its state vector V_{op} coming from site S_{op} , this operation must not be executed by the application until all operations that precede it have been received and executed on S . The procedure that undertakes the reception is the following.

procedure receive(op, S_{op}, V_{op}):

begin

wait until $V_S[i] \geq V_{op}[i], (\forall i: 1 \leq i \leq N)$;

execute (op);

$V_S[S_{op}] := V_S[S_{op}] + 1$;

end

If the operation is generated by site S (it is a local operation), then $S_{op} = S$; in this case, the waiting condition is true and so this local operation is executed immediately. Thanks to this reception procedure the following lemmas can be proved.

Lemma 3. Evolution of the state vector.

Let S be a site whose state vector is V_S . The reception procedure guarantees that each component $V_S[j]$ of the state vector is incremented from n to $n+1$ only when site S receives and executes the operation op , generated by the site j , and verifying $V_{op}[j] = n$.

Proof. By recurrence on $V_S[j]$. Suppose that $V_S[j]=0$. Before executing an operation op sent by site j , S must verify the wait condition in the reception procedure: $V_S[j] \geq V_{op}[j]$ that is to say $0 \geq V_{op}[j]$ so $V_{op}[j] = 0$. After the execution of op , $V_S[j]$ will be incremented by 1.

Suppose that the lemma is true for $V_S[j] = n$ and prove it for $n+1$. The n operations, generated by j and executed by S , which made $V_{op}[j]$ evolve from 0 to n , are those whose state vectors were: ($V_{op0}[j] = 0, V_{op1}[j] = 1, \dots, V_{opn-1}[j] = n-1$). So, operations that have not been executed yet are those whose state vector is such as $V_{op}[j] \geq n$. Among these operations, the next one to be executed is the one that verifies $V_S[j] \geq V_{op}[j]$, that is $n \geq V_{op}[j] \geq n$. It results that $V_{op}[j] = n$. After the execution of op , $V_S[j]$ will change to $n+1$.

Lemma 4. Causal precedence between two operations.

Given two messages $\langle op_1, S_{op1}, V_{op1} \rangle$ and $\langle op_2, S_{op2}, V_{op2} \rangle$, we have:

op_1 precedes op_2 iff $V_{op1}[S_{op1}] < V_{op2}[S_{op1}]$.

Proof. Necessary condition: if op_1 precedes op_2 then op_1 was received and executed on S_{op2} before the execution of op_2 . According to lemma 3, after the execution of op_1 on S_{op2} , $V_{S_{op2}}[S_{op1}]$ will have the value $V_{op1}[S_{op1}] + 1$. It results that $V_{op1}[S_{op1}] < V_{op2}[S_{op1}]$.

Sufficient Condition: if $V_{op1}[S_{op1}] < V_{op2}[S_{op1}]$ then, when op_2 was generated, the state vector of site S_{op2} , verified $V_{op1}[S_{op1}] < V_{S_{op2}}[S_{op1}]$. The value of the component $V_{S_{op2}}[S_{op1}]$ has been incremented from $V_{op1}[S_{op1}]$ to $V_{op1}[S_{op1}] + 1$; in particular, this means, according to lemma 3, that op_1 has already been executed on site S_{op2} . Consequently, since op_1 was executed before the generation of op_2 then op_1 precedes op_2 .

Lemma 5. Causal reception. Suppose two messages $\langle op_1, S_{op_1}, V_{op_1} \rangle$ and $\langle op_2, S_{op_2}, V_{op_2} \rangle$. On each site S , the reception procedure guarantees that if op_1 precedes op_2 then op_1 will be executed before op_2 .

Proof. Suppose that on site S , op_2 was received before op_1 . Since op_1 has not yet been executed, then according to lemma 3, the value of $V_S[S_{op_1}]$ cannot be higher than $V_{op_1}[S_{op_1}]$, so $V_S[S_{op_1}] \leq V_{op_1}[S_{op_1}]$. As op_1 precedes op_2 , then $V_{op_1}[S_{op_1}] < V_{op_2}[S_{op_1}]$. It results that $V_S[S_{op_1}] < V_{op_2}[S_{op_1}]$. Consequently, the condition $V_S[i] \geq V_{op_2}[i]$ ($\forall i : 1 \leq i \leq N$) in the reception procedure is not satisfied, and the operation op_2 is not executed. So, op_2 cannot be executed before op_1 .

In conclusion, causal reception guarantees that when one operation op is received and executed on a site S , then:

- all the operations which precede op have already been executed on this site,
- op does not precede any operation executed before it.

Lemma 6. Concurrent operations.

Let $\langle op_1, S_{op_1}, V_{op_1} \rangle$ and $\langle op_2, S_{op_2}, V_{op_2} \rangle$ be two messages received by site S such that the execution of op_1 on S preceded that of op_2 . Then, op_1 and op_2 are concurrent iff $V_{op_1}[S_{op_1}] \geq V_{op_2}[S_{op_1}]$.

Proof. The inequality $V_{op_1}[S_{op_1}] \geq V_{op_2}[S_{op_1}]$ implies, according to lemma 4, that op_1 does not precede op_2 . But according to lemma 5, op_2 does not precede op_1 since op_1 was executed before op_2 . So, op_1 and op_2 are concurrent.

3.3 Construction of Equivalent Local Orders in Presence of Concurrent Operations

Case of Two Concurrent Operations

Consider two concurrent operations op_1 and op_2 , respectively generated by sites S_1 and S_2 from the state x (Fig. 5). When site S_1 receives op_2 , it cannot execute this operation on the state x because the execution of op_1 has changed the object state to y . So, op_2 must be transposed forward with op_1 so as to obtain an operation defined on y that respects the intention of the user of site S_2 . It is the same for op_1 which must be transposed forward with op_2 on site S_2 .

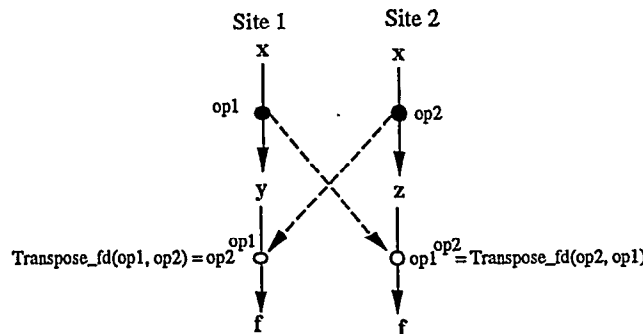


Figure 5. Forward Transposition.

In order to keep both object copies consistent, it is necessary, in this example, that $x.op_1.op_2^{op_1}$ be equal to $x.op_2.op_1^{op_2}$. More generally, to guarantee the consistency

of the copies, the forward transposition must verify the following condition.

Condition 1. Let op_1 and op_2 be two operations such as:

$$op_2^{op_1} = \text{Transpose_fd}(op_1, op_2) \text{ and}$$

$$op_1^{op_2} = \text{Transpose_fd}(op_2, op_1)$$

then $op_1.op_2^{op_1} \equiv op_2.op_1^{op_2}$.

As $op_1.op_2^{op_1} = op_1.op_2$ and $op_2.op_1^{op_2} = op_2.op_1$, condition 1 can also be written under the following form: $op_1.op_2 \equiv op_2.op_1$.

The forward transposition functions for the operations insert and delete previously defined verify condition 1 except when two insertion operations insert two different characters at the same position. In this case, a choice has to be made: which character must be inserted before the other? The solution that we have adopted consists in associating a priority to each character (for example, the character's code). In case of a conflict, the character whose priority $pr(c)$ is the highest is inserted before the other. Taking this modification into account, the definition of Transpose_fd for these two operations becomes:

$\text{Transpose_fd}(\text{insert}(p_1, c_1), \text{insert}(p_2, c_2)) =$
case $p_1 ? p_2$ of
 $p_1 < p_2 : \text{return insert}(p_2 + 1, c_2);$
 $p_1 > p_2 : \text{return insert}(p_2, c_2);$
 $p_1 = p_2 : \text{if } c_1 = c_2 \text{ then return id}$
 $\text{else if } pr(c_2) > pr(c_1) \text{ then return insert}(p_2, c_2)$
 $\text{else return insert}(p_2 + 1, c_2); \text{endif}$
endif

endcase

The part inside the frame contains the modifications added to the definition given in the paragraph 2.2.

Generalization to n Concurrent Operations

Suppose now that n sites generate, from the same state O_i of the object O , n concurrent operations $op_1, op_2, \dots, op_n$ realizing respectively the n intentions $I_1, I_2, \dots, I_n$. These operations are broadcast to all sites. They may be received in any order. If they arrive at a site S in the order $op_1, op_2, \dots, op_n$ then the sequence executed on this site will be $(op_1.op_2 : \dots.op_n)$. This sequence respects the series of intentions $I_1.I_2 \dots I_n$. The problem is then to satisfy the consistency property, so as to guarantee the consistency of the copies.

Consistency Property. Let $(op_1, op_2, \dots, op_n)$ be n operations generated in parallel and respecting the intentions $(I_1, I_2, \dots, I_n)$. Let f be an integer function $[1..n] \rightarrow [1..n]$. The consistency property expresses that: the realization of $[I_{f(1)}, I_{f(2)}, \dots, I_{f(n)}]$ is equivalent to the realization of $[I_1, I_2, \dots, I_n]$. In other words:

$$op_{f(1)}.op_{f(2)} : \dots.op_{f(n)} \equiv op_1 : op_2 : \dots.op_n.$$

As seen previously, condition 1 is necessary to guarantee that the realization of $I_1.I_2$ is equivalent to that of $I_2.I_1$. The next example shows that this condition is not sufficient to prove the consistency property in the general case of n operations.

Example 3. Suppose that three users on three sites see the word "buver" and they want to change it to "buffer". So:

1. the first user adds 'f' after the 'u': $op_1 = \text{insert}(3, 'f')$;
2. the second user deletes the 'v': $op_2 = \text{delete}(3)$;
3. the third user adds 'f' after the 'v': $op_3 = \text{insert}(4, 'f')$.

These operations are broadcast to all sites, then received and executed after forward transposition as shown in figure 6.

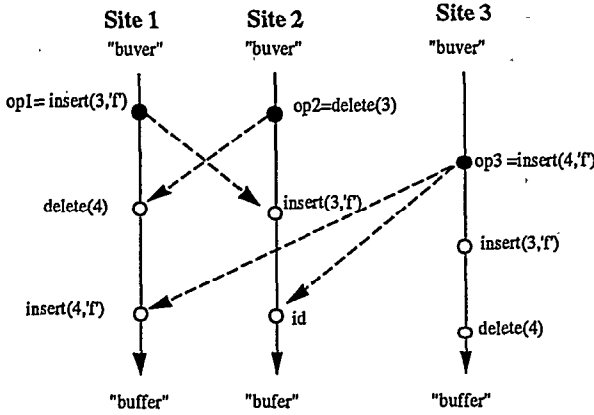


Figure 6. Consistency property is not respected.

On site 1, the local operation $\text{insert}(3, 'f')$ is executed. Then $\text{delete}(3)$ is received; it is transposed forward with $\text{insert}(3, 'f')$ and becomes $\text{delete}(4)$. When the operation $\text{insert}(4, 'f')$ arrives, it is transposed forward with $\text{insert}(3, 'f')$, then with $\text{delete}(4)$, so it becomes $\text{insert}(4, 'f')$. The word finally obtained is "buffer".

On site 2, the local operation $\text{delete}(3)$ is executed, then $\text{insert}(3, 'f')$ is received. It is not modified by the forward transposition with $\text{delete}(3)$. The last received operation $\text{insert}(4, 'f')$ is transposed forward with $\text{delete}(3)$ and becomes $\text{insert}(3, 'f')$; it is then transposed forward with $\text{insert}(3, 'f')$ which gives the null operation (id). The word finally obtained is "bufer" which is different from that obtained on site 1.

So, the objective is now to determine a sufficient condition to be verified by the function Transpose_fd in order to guarantee the consistency property.

When analysing the previous example, we notice that the histories on site 1 and site 2 are not equivalent. To obtain the equivalence, the transposition of the third operation $\text{insert}(4, 'f')$ should be the same on site 1 and on site 2. In other words, the forward transposition must give:

$$\text{insert}(4, 'f')\text{insert}(3, 'f')\text{delete}(3) = \text{insert}(4, 'f')\text{delete}(3)\text{insert}(3, 'f')$$

This means that the forward transposition of one operation with a sequence of two operations does not depend on the order in which the operations of the sequence are transposed. This constraint is expressed by Condition 2.

Condition 2. Whatever the operations op_i, op_j et op_k be, Transpose_fd must verify:

$$op_k^{op_i:op_j} = op_k^{op_j:op_i}$$

We have proved that conditions 1 and 2 are sufficient to guarantee the consistency property. We have expressed these results by the following theorem. The proof will not be presented here due to lack of space.

Consistency Theorem. If whatever the concurrent operations op_i, op_j et op_k are, the function Transpose_fd verifies the following conditions:

Condition 1: $[op_i:op_j] = [op_j:op_i]$,

Condition 2: $op_k^{op_i:op_j} = op_k^{op_j:op_i}$,

then: $\forall n$ concurrent operations $op_1, op_2, \dots, op_n$ and

$\forall f$ an integer functions $[1-n] \rightarrow [1-n]$:

$$op_{f(1)}:op_{f(2)}:\dots:op_{f(n)} \equiv op_1:op_2:\dots:op_n$$

After studying and evaluating both possibilities $op_k^{op_i:op_j}$ and $op_k^{op_j:op_i}$ for every triplet of operations $(op_i, op_j, op_k) \in \{\text{insert}, \text{delete}\}^3$, it appears that the problem comes from the conflict between two insertion operations of a character at the same position. To solve this kind of conflict, we need to know how these operations were generated. Two cases are possible: either two users have inserted two characters at the same position, or two users have inserted the two characters at two different positions and a third user has concurrently deleted characters between these positions.

To determine the correct case, two additional parameters (b and a) are added to each insertion operation. b , (resp. a) is a set of the operations that have deleted a character *before* (resp. *after*) the inserted character. So, for two operations $\text{insert}(p, c_1, b_1, a_1)$ and $\text{insert}(p, c_2, b_2, a_2)$, defined on the same state, we have one of the following cases:

- $b_1 \cap a_2 \neq \emptyset \Rightarrow c_2$ was inserted before c_1 .
- $a_1 \cap b_2 \neq \emptyset \Rightarrow c_2$ was inserted after c_1 .
- $b_1 \cap a_2 = a_1 \cap b_2 = \emptyset \Rightarrow c_1$ and c_2 were inserted at the same position.

So, Transpose_fd must be modified as follows:

$\text{Transpose_fd}(\text{insert}(p_1, c_1, b_1, a_1), \text{insert}(p_2, c_2, b_2, a_2)) =$

case $p_1 ? p_2$ of

$p_1 < p_2$: return $\text{insert}(p_2 + 1, c_2, b_2, a_2)$;

$p_1 > p_2$: return $\text{insert}(p_2, c_2, b_2, a_2)$;

$p_1 = p_2$:

if $(a_1 \cap b_2 \neq \emptyset)$ then return $\text{insert}(p_2 + 1, c_2, b_2, a_2)$

else if $(b_1 \cap a_2 \neq \emptyset)$ then return $\text{insert}(p_2, c_2, b_2, a_2)$

else if $pr(c_2) > pr(c_1)$ then return $\text{insert}(p_2, c_2)$

else if $pr(c_2) < pr(c_1)$ then return $\text{insert}(p_2 + 1, c_2)$

else return id();

endcase

$\text{Transpose_fd}(\text{delete}(p_1), \text{insert}(p_2, c_2, b, a)) =$

if $p_1 < p_2$ then return $\text{insert}(p_2 - 1, c_2, b + \text{delete}(p_1), a)$

else return $\text{insert}(p_2, c_2, b, a + \text{delete}(p_1))$; endif

3.4 Integration of an Operation in the History

Let $H_S(n) = op_1.op_2.\dots.op_n$ be the history of the object O on the site S . O_i (resp. O_f) is the initial (resp. current) state of that object. Let op be an operation sent by S' and received by S . This operation was generated from the state O_j in order to realize the intention $I = \text{Intention}(op, O_j)$. To integrate op in $H_S(n)$ means determining and executing on the current state O_f an operation op_{n+1} that respects I , that is $\text{Intention}(op_{n+1}, O_f) = \text{Intention}(op, O_j) = I$.

Due to the reception procedure, the history $H_S(n)$ verifies:

- all operations that precede op are integrated in $H_S(n)$;
- op does not precede any operation of $H_S(n)$.

The causality is thus respected, and it remains to transpose op forward with its concurrent operations to obtain an operation whose execution respects the user intention and guarantees the data consistency. To achieve this, the naive method consists of going through the history $H_S(n)$ and transposing op forward with each operation concurrent to it [4]. This method is implemented by the following algorithm.

Notations:

op : received operation

S_{op} : op's generator site

V_{op} : state vector associated to op

H : representation of $H_S(n)$ = array of n elements of the form $\langle op_i, S_{op_i}, V_{op_i} \rangle$

n : number of elements in H .

procedure Integrate ($\langle op, S_{op}, V_{op} \rangle$):

begin

for $i := 1$ **to** n **do**

$\langle op_i, S_{op_i}, V_{op_i} \rangle := H[i];$

if $V_{op_i}[S_{op_i}] \geq V_{op}[S_{op_i}]$ **then**

$\{ op_i // op \text{ according to lemma 6} \}$

$op := \text{Transpose_fd}(op_i, op);$ **endif**

endfor

execute (op);

$H[n+1] := \langle op, S_{op}, V_{op} \rangle;$

$n := n + 1;$

end

Partial Concurrency Problem.

The previous algorithm produces in some cases inconsistent results characterised by the divergence of the copies. These cases correspond to a partial concurrency situation evoked in [1]. The following example shows this situation.

Example 4. Consider two users who try to change the word "telephone" into "telephone". The script is as follows (Fig. 7): the first user puts the cursor before the 'f' and types 'ph'; this action is translated into two operations $op_1 = \text{insert}(5, 'p')$ and $op_2 = \text{insert}(6, 'h')$. At the same instant, the second user deletes the 'f': $op_3 = \text{delete}(5)$. So, op_1 precedes op_2 , and op_3 is concurrent to op_1 , but op_3 is partially concurrent to op_2 insofar as op_2 and op_3 are not generated from the same state.

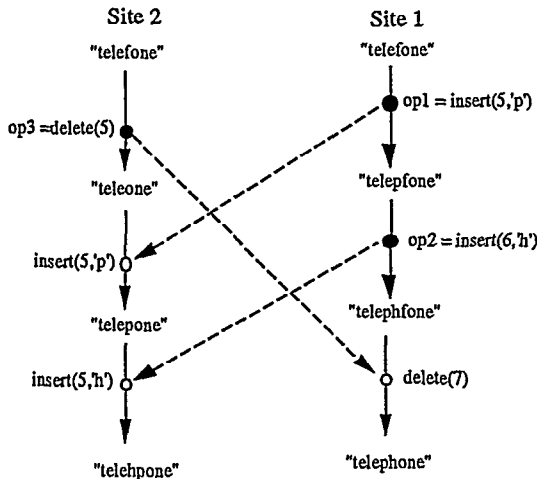


Figure 7. Partial concurrency between operations.

On site 1, when op_3 is received, it is transposed forward with op_1 then with op_2 . The resulting operation $op_3' = \text{delete}(7)$. The execution of this operation gives the wanted word "telephone". On site 2, the transposition of op_1 with op_3 gives $op_1' = op_1 = \text{insert}(5, 'p')$. But the transposition of op_2 with op_3 gives $op_2' = \text{insert}(5, 'h')$ whose execution gives the word "telephne" which is different from what is obtained on site 1.

This result, which does not correspond to the user intention, is due to a wrong application of the forward transposition function on two operations partially concurrent (op_2 and op_3 on site 2). This function requires that both operations are defined on the same state. This was not the case on site 2 where $\text{delete}(5)$ was generated from the state "telephone" while $\text{insert}(6, 'h')$ was generated from the word "telephne". For that reason the forward transposition of $\text{insert}(6, 'h')$ with $\text{delete}(5)$ gave a wrong result. In order to solve the partial concurrency problem, the history must be re-ordered so that the forward transposition can be applied correctly.

Let op be an operation to be integrated into $H_S(n)$. Let $H_S(m)$ be the history of the site where op was generated, taken at that instant. If O_i was the initial state, then op was generated from the state: $O_i, H_S(m) = O_m$. Notice that all the operations of $H_S(m)$ precede op and that all these operations were already in $H_S(n)$. This gives us the idea to reorder $H_S(n)$ in a way that its operations are regrouped into two sequences seq_1 and seq_2 such as: $H_S(n) \equiv seq_1.seq_2$ with:

$seq_1 \equiv H_S(m)$ = the sequence of operations that precede op ,

seq_2 = the sequence of operations concurrent to op .

Under these conditions, seq_2 is defined on the state $O_i, seq_1 = O_i, H_S(m) = O_m$. Consequently, seq_2 and op are defined on the same state. This enables lemma 1 to be applied to obtain: $\text{Intention}(op, O_m) = \text{Intention}(op^{seq_2}, O_m.seq_2)$ which gives: $H_S(n+1) \equiv seq_1.seq_2.op^{seq_2}$.

The problem that now needs to be solved is the following: how to reorder $H_S(n)$ into two sequences without violating the intentions of the users. To do this, we will use another type of transposition called *backward transposition*. This transposition was introduced in [15] to undo an operation in a history without undoing the operations that follow it.

Backward Transposition

Suppose that on a site S , a history $op_1.op_2$ realizes $I_1.I_2$. Changing the execution order of op_1 and op_2 means finding the history $op_2'.op_1'$ that would have existed if op_2 had been executed before op_1 . This history would have realized $I_2.I_1$.

We will define a function that enables the execution order between two operations to be exchanged without violating their intentions. This function is called *backward transposition*, noted Transpose_bk , because it enables an operation to be moved back to the past and to simulate it being executed first.

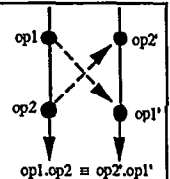
Definition 10 Backward transposition.

Let $op_1.op_2$ be a sequence of two operations.

$\text{Transpose_bk}(op_1, op_2) = op_2'.op_1'$ with:

$op_2' = \text{Transpose_fd}(op_1, op_2)$ and

$op_1' = \text{Transpose_fd}(op_2', op_1)$.



It results from this definition that, as *Transpose_fd* verifies condition 1, *Transpose_bk* verifies the following property:

if *Transpose_bk* (op_1, op_2) = $op_2'.op_1'$ then

$$op_1.op_2 \equiv op_2'.op_1'.$$

Transpose_bk is specified as *Transpose_fd* for every couple of operations ($op_1.op_2$). For example, the function *Transpose_bk* for the couple (*insert*(p_1, c), *delete*(p_2)) is defined as follows:

```

Transpose_bk (insert( $p_1, c$ ), delete( $p_2$ )) =
  case  $p_1 ? p_2$  of
     $p_1 < p_2$ : return (delete( $p_2 - 1$ ). insert( $p_1, c$ ));
     $p_1 > p_2$ : return (delete( $p_2$ ). insert( $p_1 - 1, c$ ));
     $p_1 = p_2$ : return (id. id);
  endcase

```

We leave to the readers to determine the backward transpositions for the other couples of operations.

Reordering $H_S(n)$ into Two Sequences

Let $H_S(n) = op_1.op_2....op_i.op_{i+1}....op_n$ be a history that respects the intentions $I_1.I_2...I_i.I_{i+1}....I_n$. Let $op_{i+1}.op_i' = \text{Transpose_bk}(op_i, op_{i+1})$. Since $op_{i+1}.op_i' \equiv op_i.op_{i+1}$, the execution order of op_i and op_{i+1} in $H_S(n)$ could be changed without touching the other operations or violating their intentions; this gives $H_S(n) \equiv op_1.op_2...op_{i+1}.op_i'....op_n$ that respects the intentions $I_1.I_2....I_{i+1}.I_i....I_n$.

Every application of the backward transposition enables two operations to be permuted. It is thus possible to reorder the history in any order. To do this, it is sufficient to know the necessary permutations, then to apply the transpositions that correspond to these permutations. For our problem, $H_S(n)$ must be reordered so that the operations which precede *op*, the operation to be integrated, are regrouped before the operations which are concurrent to *op*. In order to do this, *Transpose_bk* must be applied to each couple ($op_i.op_j$) so that op_j precedes *op*, and op_i is concurrent to *op*. This must be repeated until all the operations that precede *op* are located before the operations which are concurrent to *op*. In this way, $H_S(n)$ will be separated into two sequences seq_1 and seq_2 , so that all the operations of seq_1 precede *op* and all the operations of seq_2 are concurrent to *op*.

Algorithm

The procedure *Transpose_backward* redefines the function *Transpose_bk* so that it transposes backward two operations in the history $H_S(n)$. The function *Separate* separates the history $H_S(n)$ into two sequences seq_1 and seq_2 , and returns the length n_1 of seq_1 . The procedure *Integrate* transposes forward the operation *op* with the operations of seq_2 then it executes the resulting operation. This procedure is called by the reception procedure presented in 3.2.

Notations:

- op* : received operation
- S_{op} : *op*'s generator site
- V_{op} : state vector associated to *op*
- H : representation of $H_S(n)$ = array of n elements of the form $\langle op_i, S_{op_i}, V_{op_i} \rangle$
- n : number of elements in H
- n_1 : length of seq_1 calculated by the procedure *Separate*.

procedure Transpose_backward(j) : {Transpose backward the j^{th} operation with the $(j-1)^{th}$ operation}

begin

```

 $\langle op_j, S_{op_j}, V_{op_j} \rangle := H[j];$ 
 $\langle op_{j-1}, S_{op_{j-1}}, V_{op_{j-1}} \rangle := H[j-1];$ 
 $op_j.op_{j-1} = \text{Transpose\_bk}(op_{j-1}, op_j);$ 
 $H[j] := \langle op_{j-1}, S_{op_{j-1}}, V_{op_{j-1}} \rangle;$ 
 $H[j-1] := \langle op_j, S_{op_j}, V_{op_j} \rangle;$ 

```

end

function Separate ($\langle op, S_{op}, V_{op} \rangle$);

begin

```

 $n_1 := 0;$ 
for  $i := 1$  up to  $n$  do
   $\langle op_i, S_{op_i}, V_{op_i} \rangle := H[i];$ 
  if  $V_{op_i}[S_{op_i}] < V_{op}[S_{op}]$  then {op precedes op}
    for  $j := i$  down to  $n_1 + 2$  do
      {transpose opi backward to seq1}
      Transpose_bk (j);
    endfor;
  endif;
   $n_1 := n_1 + 1;$ 
endfor;
return ( $n_1$ );

```

end

procedure Integrate ($\langle op, S_{op}, V_{op} \rangle$);

begin

```

 $n_1 := \text{Separate}(\langle op, S_{op}, V_{op} \rangle);$ 
for  $i := n_1 + 1$  up to  $n$  do {transpose op forward with regard to seq2}
   $\langle op_i, S_{op_i}, V_{op_i} \rangle := H[i];$ 
   $op := \text{Transpose\_fd}(op_i, op);$ 
endfor;
execute (op);
 $n := n + 1;$ 
 $H[n] := \langle op, S_{op}, V_{op} \rangle;$ 
end.

```

Let us see an illustration of this algorithm in example 4.

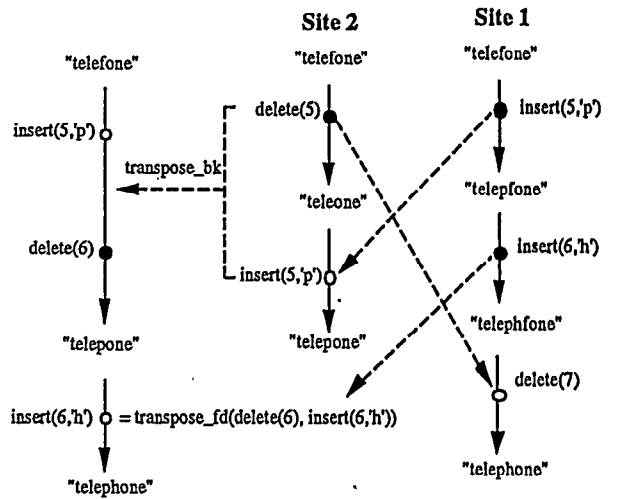


Figure 8. Solution for the partial concurrency.

The purpose is to integrate the operation *insert*(6,'h') in the history of site 2: $H(n) = \text{delete}(5).\text{insert}(5, 'p')$. In this history (Fig. 8), *insert*(5,'p') is the only operation that precedes the operation *insert*(6,'h'). So, to separate $H(n)$ into two

sequences seq₁ and seq₂, both operations of H(n) must be transposed backward. This gives:

$H(n) = \text{Transpose_bk}(\text{delete}(5), \text{insert}(5, 'p')) = \text{insert}(5, 'p'). \text{delete}(6)$

$\text{seq}_1 = \text{insert}(5, 'p')$ and $\text{seq}_2 = \text{delete}(6)$.

It is clear that delete(6) and insert(6, 'h') are now defined on the same state, (corresponding to the word "telephone" obtained after the execution of insert(5, 'p') from the word "telephone"). When transposing forward insert(6, 'h') with delete(6), the results is the operation insert(6, 'h') to be executed on the word " telephone". Thus, the final word obtained is "telephone".

4. CONCLUSION

We propose a new approach for concurrency control in collaborative systems. This approach enables the history associated with an object to be constructed incrementally, without ever undoing and redoing old operations. It is based on the notion of intention, that enables equivalent histories to be characterised. Formally, this equivalence is obtained by semantic properties of operations that we call forward transposition and backward transposition. Forward transposition resolves the problem of concurrent operations, whilst backward transposition changes the order of operations in the history while preserving equivalence.

Implementation in a centralised and then in a distributed environment is proposed. The distributed model seems to be very well adapted to mobile systems in which the reconnection of a site requires old operations to be integrated into the history of each site.

5. REFERENCES

1. Allison C.; "Concurrency Control for Real Time Groupware"; CE94, Concurrent Engineering Research and Applications. Aglobal Perspective, pp. 163-170, CTC Pittsburg August 1994.
2. Cart M., Ferrié J., Richy H.; "Contrôle de l'exécution de transactions concurrentes"; Technique et Science Informatique. Vol 8, N 3, 1989. pp 225-240.
3. Decouchant D., Quint V., Vatton I.; "L'édition Coopérative de documents avec Griffon"; Colloque IHM'92, num. Paris, Decembre 1992.
4. Ellis C.A., Gibbs S.J.; "Concurency Control in Groupware Systems"; In Proceedings of the ACM SIGMOD'89 Conference on the Management of Data (Seattle, May ACM 1989, pp.399-407.
5. Ellis C.A., Gibbs S.J., Rein G.L.; "Groupware: Some issues and experiences"; Communication of the ACM Jan 1991 Vol.34, No.1, P39-59.
6. Fish R.S., Kraut R.E., Leland M.D.P., Cohen M; "Quilt: a collaborative tool for cooperative writing"; In Proceedings of the conference on Office Information Systems, (Palo Alto 23-25 Mars), ACM, Jun 1988, pp. 30-37.
7. Greenberg S., Roseman M., Webster D., Bohnet R.; "Issues and Experiences Designing and Implementing Tow Group Drawing Tools"; IEEE 1992, Proc. of Hawaii Inter. Conf. on System Sciences. 4, pp 139-150.
8. Greenberg S., Marwood D.; "Real Time Groupware as a Distributed System: Concurrency Control and its Effect on the Interface"; In proceedings of the ACM Conference on CSCW, North Canada, 22-26 October 1994, pp. 207-217.
9. Greif I., Seliger R., Weihi W.; "Atomic Data Abstraction in a Distributed Collaborative Editing System"; In Proceedings of the 13th annual Symposium on Principles of Programming Languages. Petersburg, ACM, Jan. 1986, pp. 160-172.
10. Hawryszkiewicz I., Karagiannes D., Maciaszek L., Teufel B.; "Respons: Requirements Specific Object Model for Workgroup Computing"; International Journal of Intelligent And Cooperative information Systems; Vol.3, No.3 (1994) pp. 293-318.
11. Karsenty A., Beaudouin-Lafon M.; "An Algorithm for Distributed Groupware Applications"; 13th conf. on Distributed Groupware Computing Systems ICDCS'93 Pittsburgh, IEEE, may 1993, pp.195-202.
12. Kanawati R.; "Replicated Data Management Algorithm for Distributed Synchronous Groupware Applications". DAPSYS'96 (environnements and tools), Miskloc, Hungary. October 1996.
13. Lamport L.; "Time, Clocks, and the Ordering of Events in Distributed System", Communications of the ACM, 1978, vol. 21, N. 7, p. 558, 565.
14. Mattern F.; "Virtual time and global states of Distributed Systems"; Proc. Int. Workshop on Dist. and Par. Alog. North-Holland (1989), p. 215-276.
15. Prakash A., Knister M.J.; "Undoing Actions in Collaborative Work"; CSCW'92 Proceedings, November 1992. pp. 273-280.
16. Prakash A., Shim H. S.; "DistView: Support for Building Efficient Collaborative Applications using Replicated Object"; ACM conference on CSCW, ACM Press October 1994. pp. 153-164.
17. Raynal M.; "La communication et le temps dans les réseaux et les systèmes répartis"; Eyrolles, Collection DER-EDF, 1991.
18. Sun C., Yang Y., Zhang Y., Chen D.; "Distributed Concurrency Control in Real-time Cooperative Editing Systems", 2nd Asian Computing Science Conf. on Concurrency and Parallelism, Programming, Networking, Singapour, Dec. 1996. pp.84-95
19. Traumuller R.; "Computer Supported Cooperative Work (CSCW)"; GMD-Studien - Nr. 241, September 1994. pp. 37-49.
20. Weihi W.E.; "Commutativity-Based Concurrency Control for Abstract Data Type"; IEEE Transactions On Computers, Vol 37, No. 12 December 1988